REVISION CHEAT SHEET

CONSTRUCTORS / INITIALIZATION

Constructors and Object Initialization in C#

Interview-ready notes · examples · C# · diagrams

Topic 12 of 40

Constructors are one of the most important parts of object-oriented C# because they control **how an object enters the application**.

A class may contain excellent validation methods and encapsulated properties, but if callers can create the object in an invalid state, the design is already weak.

Consider a fundraising `Gift`.

A valid gift may require:

EXAMPLE / FLOW

```
GiftId
+
TenantId
+
ConstituentId
+
TotalAmount > 0
+
CurrencyCode
```

Ideally:

EXAMPLE / FLOW

```
new Gift(...)
    →
validation
    →
initialization
    →
valid Gift object
```

rather than:

EXAMPLE / FLOW

```
new Gift()
    →
half-initialized object
    →
caller remembers to set property 1
    →
caller remembers property 2
    →
caller forgets property 3
    →
invalid object
```

A constructor helps establish the object's initial state.

This topic covers:

- what constructors are;
- why constructors exist;
- constructor syntax;
- constructor vs ordinary method;
- parameterless constructors;
- compiler-generated default constructors;
- explicit parameterless constructors;
- parameterized constructors;
- constructor parameters;
- constructor validation;
- overloaded constructors;
- constructor overloading rules;
- constructor chaining;
- `this(...)`;
- avoiding duplicated initialization;
- `base(...)`;
- implicit `base()`;
- constructor execution through inheritance;
- constructor accessibility;
- public constructors;
- private constructors;
- protected constructors;
- internal constructors;
- static constructors;
- static field initialization;
- constructor exceptions;
- expression-bodied constructors;
- fields and property initializers;
- object initializers;
- `init`;
- `required`;
- `SetsRequiredMembers`;
- initialization order;
- primary constructors;
- primary-constructor storage;
- primary constructors and dependency injection;
- constructor design best practices;
- constructor side effects;
- ASP.NET Core dependency injection;
- Fundraising Platform examples;
- performance considerations;
- security considerations;
- current C# 14 partial constructors;
- interview questions;
- practical exercises.

The central idea is:

EXAMPLE / FLOW

```
Object Requested
    →
Constructor Selected
    →
Required Inputs Provided
    →
Object State Initialized
    →
Rules Validated
    →
Usable Object Returned
```

Microsoft defines a constructor as a member invoked by the runtime when an instance of a class or struct is created. Constructors allow a type to ensure its instances start in a valid state.

What Is a Constructor?

Definition

A **constructor is a special member of a class or struct that runs during object creation and is normally responsible for initializing the new instance**.

Example:

C#

```
public sealed class Gift
{
    public Gift()
    {
        Console.WriteLine(
            "Gift created.");
    }
}
```

Create an object:

C#

```
Gift gift = new Gift();
```

Flow:

EXAMPLE / FLOW

```
new Gift()
    →
runtime creates object
    →
Gift constructor runs
    →
constructor completes
    →
Gift reference returned
```

Constructor Syntax

Basic syntax:

```C#
public Gift()
{
}
```

Notice several important things:

EXAMPLE / FLOW

```
Constructor name
    =
Class name
```

and:

EXAMPLE / FLOW

```
No return type
```

Not even:

EXAMPLE / FLOW

```
void
```

This Is a Constructor

```C#
public Gift()
{
}
```

This Is a Method

```C#
public void Gift()
{
}
```

The second example has:

EXAMPLE / FLOW

```
void
```

so it is an ordinary method named `Gift`.

It is **not** a constructor.

Constructor vs Method

Constructor	Method
Same name as containing type	Can have almost any valid method name
No return type	Has return type or `void`
Runs during object construction	Runs when explicitly called
Initializes instance	Performs general behavior

Constructor	Method
Invoked through `new`/constructor chaining	Invoked with method-call syntax
Cannot return a value	Can return values

Why Constructors Exist

Without a constructor, callers might create an invalid object.

Example:

```C#
public sealed class Gift
{
    public decimal TotalAmount
    {
        get;
        set;
    }

    public string? CurrencyCode
    {
        get;
        set;
    }
}
```

Then:

```C#
Gift gift = new Gift();
```

produces something conceptually like:

```EXAMPLE / FLOW
TotalAmount = 0

CurrencyCode = null
```

But if a valid gift must always have:

```EXAMPLE / FLOW
Amount > 0
+
Currency
```

then that object should not be considered valid.

Constructor-Enforced Initialization

Better:

```C#
public sealed class Gift
{
    public Gift(
        decimal totalAmount,
        string currencyCode)
    {
```

```
        if (totalAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
            currencyCode);

        TotalAmount = totalAmount;

        CurrencyCode = currencyCode;
    }

    public decimal TotalAmount
    {
        get;
    }

    public string CurrencyCode
    {
        get;
    }
}
```

Now:

C#
```
Gift gift =
    new Gift(
        1000m,
        "USD");
```

creates a valid object.

But:

C#
```
new Gift(
    -500m,
    "USD");
```

fails immediately.

Constructor as a Valid-State Boundary

A useful design principle is:

> ⓘ If an object cannot meaningfully exist without some information, consider requiring that information during construction.

Flow:

EXAMPLE / FLOW
```
Inputs
    →
Constructor
    →
Validate
    →
Invalid?
```

```
    →
Throw / reject creation

Valid?
    →
Assign state
    →
Return valid object
```

Parameterless Constructor

Definition

A constructor that takes no parameters is called a:

EXAMPLE / FLOW

```
parameterless constructor
```

Example:

C#

```
public Gift()
{
}
```

Usage:

C#

```
Gift gift = new Gift();
```

Microsoft uses this terminology directly: a constructor with no parameters is a parameterless constructor.

Compiler-Generated Parameterless Constructor

Suppose:

C#

```
public class Gift
{
}
```

and you don't declare any instance constructors.

For a normal class, the C# compiler supplies a public parameterless constructor.

Therefore this works:

C#

```
Gift gift = new Gift();
```

The compiler supplies that constructor only when you haven't declared an instance constructor yourself.

Conceptual Compiler Behavior

You write:

```C#
public class Gift
{
}
```

Conceptually similar to:

```C#
public class Gift
{
    public Gift()
    {
    }
}
```

for construction purposes.

Important Rule

Once you declare your own instance constructor:

```C#
public class Gift
{
    public Gift(
        decimal amount)
    {
    }
}
```

the compiler does **not** additionally create:

```C#
public Gift()
{
}
```

for you.

Example

```C#
public sealed class Gift
{
    public Gift(
        decimal amount)
    {
        Amount = amount;
    }

    public decimal Amount
    {
        get;
    }
}
```

Valid:

```C#
Gift gift =
    new Gift(
        1000m);
```

Invalid:

```C#
// Gift gift = //     new Gift();
```

because no parameterless constructor exists.

If You Need Both

Declare both explicitly:

```C#
public sealed class Gift
{
    public Gift()
    {
    }

    public Gift(
        decimal amount)
    {
        Amount = amount;
    }

    public decimal Amount
    {
        get;
        private set;
    }
}
```

Now both are valid:

```C#
Gift first = new Gift();

Gift second =
    new Gift(
        1000m);
```

Should Every Class Have a Parameterless Constructor?

No.

Do not add:

```C#
public Gift()
{
}
```

only because:

EXAMPLE / FLOW
"It's common"

Ask:

EXAMPLE / FLOW
Can Gift be valid
without constructor inputs?

If answer is:

EXAMPLE / FLOW
No

then allowing:

```C#
new Gift()
```

may weaken the model.

Parameterized Constructor

Definition

A constructor with one or more parameters is a:

EXAMPLE / FLOW
parameterized constructor

Example:

```C#
public Gift(
    long constituentId,
    decimal amount)
{
    ConstituentId = constituentId;

    TotalAmount = amount;
}
```

Usage:

```C#
Gift gift =
    new Gift(
        constituentId: 5001,
        amount: 1000m);
```

Constructor parameters force callers to supply values needed during object creation.

Constructor Parameters Work Like Method Parameters

You can use:

```
Value parameters

Reference types

Value types

Optional parameters

Named arguments
```

where appropriate.

Example:

C#
```
public Gift(
    long constituentId,
    decimal amount,
    string currencyCode)
{
}
```

Call with positional arguments:

C#
```
new Gift(
    5001,
    1000m,
    "USD");
```

or named arguments:

C#
```
new Gift(
    constituentId: 5001,
    amount: 1000m,
    currencyCode: "USD");
```

Named arguments can improve readability when several parameters have similar types.

Constructor Parameter Validation

Constructors are often an excellent place to reject invalid initial state.

Example:

C#
```
public Gift(
    long constituentId,
    decimal amount,
    string currencyCode)
{
    if (constituentId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(constituentId));
    }

    if (amount <= 0m)
    {
```

```
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    ArgumentException.ThrowIfNullOrWhiteSpace(
        currencyCode);

    ConstituentId = constituentId;

    TotalAmount = amount;

    CurrencyCode =
        currencyCode
            .Trim()
            .ToUpperInvariant();
}
```

If validation fails:

EXAMPLE / FLOW

```
Object construction
    →
does not complete successfully
```

The caller receives the exception instead of a valid object reference.

Constructor Should Establish Invariants

Suppose a `GiftAllocation` must satisfy:

EXAMPLE / FLOW

```
FundId > 0

Amount > 0
```

Good:

C#

```
public sealed class GiftAllocation
{
    public GiftAllocation(
        long fundId,
        decimal amount)
    {
        if (fundId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(fundId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        FundId = fundId;

        Amount = amount;
    }

    public long FundId
```

```
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}
```

Now an allocation cannot normally be constructed with:

EXAMPLE / FLOW

```
FundId = 0
```

or:

EXAMPLE / FLOW

```
Amount = -500
```

Constructor Overloading

Definition

A class can define multiple constructors with different parameter lists.

This is called:

EXAMPLE / FLOW

```
constructor overloading
```

Example:

C#

```
public sealed class Gift
{
    public Gift()
    {
    }

    public Gift(
        decimal amount)
    {
        Amount = amount;
    }

    public Gift(
        decimal amount,
        string currencyCode)
    {
        Amount = amount;

        CurrencyCode = currencyCode;
    }

    public decimal Amount
    {
        get;
        private set;
    }

    public string CurrencyCode
```

```
    {
        get;
        private set;
    } = "USD";
}
```


Microsoft explicitly supports multiple constructors that accept different argument combinations.

Constructor Selection

Call:

```C#
new Gift()
```

selects:

```C#
Gift()
```

Call:

```C#
new Gift(
    1000m)
```

selects:

```C#
Gift(decimal amount)
```

Call:

```C#
new Gift(
    1000m,
    "EUR")
```

selects:

```C#
Gift(
    decimal amount,
    string currencyCode)
```

Constructor Overload Resolution

Constructors follow normal overload-resolution principles.

The compiler considers:

EXAMPLE / FLOW

```
Number of arguments

Argument types

Conversions

Optional parameters
```

```
Named arguments
```

to select the best accessible constructor.

If it cannot identify a unique valid constructor:

EXAMPLE / FLOW

```
compile-time error
```

Avoid Duplicate Constructor Logic

Bad:

C#

```
public Gift()
{
    Amount = 1m;

    CurrencyCode = "USD";

    Status = "Pending";
}

public Gift(
    decimal amount)
{
    Amount = amount;

    CurrencyCode = "USD";

    Status = "Pending";
}

public Gift(
    decimal amount,
    string currencyCode)
{
    Amount = amount;

    CurrencyCode = currencyCode;

    Status = "Pending";
}
```

Problem:

EXAMPLE / FLOW

```
same initialization
copied three times
```

If the business rule changes, you may forget to update every constructor.

Constructor Chaining

C# allows one constructor to call another constructor in the same class using:

EXAMPLE / FLOW

```
this(...)
```

This is called:

EXAMPLE / FLOW

```
constructor chaining
```

Example:

C#

```
public sealed class Gift
{
    public Gift()
        : this(
            1m,
            "USD")
    {
    }

    public Gift(
        decimal amount)
        : this(
            amount,
            "USD")
    {
    }

    public Gift(
        decimal amount,
        string currencyCode)
    {
        Amount = amount;

        CurrencyCode = currencyCode;
    }

    public decimal Amount
    {
        get;
    }

    public string CurrencyCode
    {
        get;
    }
}
```

Microsoft documents `: this(...)` as the syntax for invoking another instance constructor in the same class.

this(...) Flow

Call:

C#

```
new Gift(
    1000m);
```

Flow:

EXAMPLE / FLOW

```
Gift(decimal)
    →
```

```
this(decimal, string)
    →
Gift(decimal, string)
    →
central initialization
    →
return
```

Why Constructor Chaining Is Useful

Instead of:

EXAMPLE / FLOW

```
Constructor A
    →
Initialization Logic A

Constructor B
    →
Duplicate Initialization Logic

Constructor C
    →
More Duplicate Logic
```

use:

EXAMPLE / FLOW

```
Constructor A ─┐
               ↓
Constructor B ─→ Main Constructor
               ↑
Constructor C ─┘
```

This creates:

EXAMPLE / FLOW

```
one initialization path
```

Fundraising Example

C#

```
public sealed class Gift
{
    public Gift(
        long constituentId,
        decimal amount)
        : this(
            constituentId,
            amount,
            "USD",
            campaignId: null)
    {
    }

    public Gift(
        long constituentId,
        decimal amount,
        string currencyCode)
        : this(
```

```
            constituentId,
            amount,
            currencyCode,
            campaignId: null)
    {
    }

    public Gift(
        long constituentId,
        decimal amount,
        string currencyCode,
        long? campaignId)
    {
        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
            currencyCode);

        ConstituentId = constituentId;

        TotalAmount = amount;

        CurrencyCode = currencyCode;

        CampaignId = campaignId;
    }

    public long ConstituentId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public string CurrencyCode
    {
        get;
    }

    public long? CampaignId
    {
        get;
    }
}
```

Now:

EXAMPLE / FLOW

```
All constructors
    →
eventually reach
```

```
    →
one validation path
```

Constructor Chaining Rule

Do not create circular constructor chains.

Invalid concept:

EXAMPLE / FLOW

```
Constructor A
    →
Constructor B
    →
Constructor A
```

The compiler rejects direct or indirect constructor recursion.

this Constructor Initializer vs this Instance Reference

These are related keywords but different usages.

Constructor chaining:

C#

```
public Gift()
    : this(
        100m)
{
}
```

Inside an instance member:

C#

```
this.TotalAmount
```

The first means:

EXAMPLE / FLOW

```
call another constructor
in this same type
```

The second means:

EXAMPLE / FLOW

```
current object instance
```

Base Constructors

Every class except:

EXAMPLE / FLOW

```
System.Object
```

has a base class.

When a derived class object is constructed, its base-class portion must also be initialized.

Example:

C#

```
public class Donation
{
    public Donation(
        decimal amount)
    {
        Amount = amount;
    }

    public decimal Amount
    {
        get;
    }
}

public class Gift :
    Donation
{
    public Gift(
        decimal amount)
        : base(
            amount)
    {
    }
}
```

The syntax:

EXAMPLE / FLOW

```
: base(...)
```

calls a constructor from the direct base class. Microsoft documents that the selected base constructor runs before the derived constructor body.

Base Constructor Flow

EXAMPLE / FLOW

```
new Gift(1000)
    →
Gift constructor selected
    →
base(1000)
    →
Donation constructor runs
    →
Gift constructor body runs
    →
Gift object ready
```

Why Base Constructor Must Run First

A derived object contains its inherited/base-class state.

Before derived behavior initializes its own portion, the base class must establish its state.

Conceptually:

EXAMPLE / FLOW

```
Gift Object
    →
Donation/Base Portion
```

```
    +
Gift-Specific Portion
```

Construction:

EXAMPLE / FLOW

```
Initialize Base Portion
    →
Initialize Derived Portion
```

Implicit base()

Suppose:

C#

```
public class Donation
{
    public Donation()
    {
    }
}

public class Gift :
    Donation
{
    public Gift(
        decimal amount)
    {
    }
}
```

Because the derived constructor does not explicitly specify a base constructor, C# behaves as though it had:

C#

```
public Gift(
    decimal amount)
    : base()
{
}
```

If the base class has an accessible parameterless constructor, it is invoked implicitly.

When Explicit base(...) Is Required

Suppose:

C#

```
public class Donation
{
    public Donation(
        long tenantId)
    {
    }
}
```

There is no parameterless constructor.

Then:

```C#
public class Gift :
    Donation
{
    public Gift(
        long tenantId)
        : base(
            tenantId)
    {
    }
}
```

must explicitly invoke a valid base constructor.

this(...) vs base(...)

`this(...)`:

EXAMPLE / FLOW

```
calls another constructor
in the same class
```

`base(...)`:

EXAMPLE / FLOW

```
calls a constructor
in the direct base class
```

Example:

```C#
public Gift()
    : this(
        100m)
{
}
```

versus:

```C#
public Gift(
    decimal amount)
    : base(
        amount)
{
}
```

Constructor Initializer Timing

The constructor initializer:

```C#
: this(...)
```

or:

```C#
: base(...)
```

runs before the current constructor body's statements.

Therefore:

C#

```
public Gift(
    decimal amount)
    : base(
        amount)
{
    Console.WriteLine(
        "Gift body");
}
```

runs conceptually:

EXAMPLE / FLOW

```
base constructor
    →
Gift body
```

Constructor Access Modifiers

Constructors can use access modifiers such as:

EXAMPLE / FLOW

```
public

private

protected

internal

protected internal

private protected
```

depending on the design.

Accessibility controls:

EXAMPLE / FLOW

```
Who is allowed
to create the object
using this constructor?
```

Public Constructor

C#

```
public Gift(
    decimal amount)
{
}
```

Normal callers that can access the class can use it.

Internal Constructor

```C#
internal Gift(
    decimal amount)
{
}
```

Only code within the same assembly can normally use it.

This can be useful when:

EXAMPLE / FLOW

```
External code should see the type
but object creation should be controlled
inside the assembly.
```

Protected Constructor

```C#
protected Gift()
{
}
```

This is useful for types intended primarily for inheritance.

Outside callers cannot normally write:

```C#
new Gift();
```

through that constructor.

Derived classes may invoke it.

Inheritance will be covered more deeply in Topic 13.

Private Constructor

Definition

A private constructor can only be called from within the containing type.

Example:

```C#
public sealed class GiftNumber
{
    private GiftNumber()
    {
    }
}
```

Outside code cannot:

```C#
new GiftNumber();
```

Microsoft specifically documents private constructors as a way to prevent normal external instantiation.

Why Use a Private Constructor?

Common uses include:

EXAMPLE / FLOW

```
Factory-controlled creation

Singleton-style designs

Utility types

Preventing invalid direct construction

Framework-specific creation patterns
```

Factory-Controlled Construction

Example:

C#

```
public sealed class GiftReference
{
    private GiftReference(
        string value)
    {
        Value = value;
    }

    public string Value
    {
        get;
    }

    public static GiftReference Create(
        int year,
        long giftId)
    {
        string value = $"GFT-{year}-{giftId}";

        return new GiftReference(
            value);
    }
}
```

Usage:

C#

```
GiftReference reference =
    GiftReference.Create(
        2026,
        1001);
```

Outside callers cannot bypass the factory using:

C#

```
new GiftReference(...)
```

Factory Method vs Constructor

Constructor:

```C#
new Gift(...)
```

naturally communicates:

EXAMPLE / FLOW

```
Create one Gift instance
```

Factory method:

```C#
Gift.Create(...)
```

can communicate additional semantics:

EXAMPLE / FLOW

```
Create

Restore

FromExisting

Parse

TryCreate

CreateDraft
```

Factories are useful when construction requires more expressive semantics.

Design patterns will cover this much more deeply later.

Constructor Overloading vs Optional Parameters

Suppose:

```C#
public Gift(
    decimal amount,
    string currencyCode = "USD")
{
}
```

Call:

```C#
new Gift(
    1000m);
```

or:

```C#
new Gift(
    1000m,
    "EUR");
```

This can sometimes replace multiple constructor overloads.

When Optional Constructor Parameters Are Useful

Good when:

EXAMPLE / FLOW

```
One value has a natural,
stable default
```

Example:

C#

```
currencyCode = "USD"
```

only if that truly matches the business default.

When Overloads Are Better

Use constructor overloads when different creation paths represent meaningfully different contracts.

Example:

EXAMPLE / FLOW

```
Create Gift from:
Constituent + Amount

vs

Restore Gift from:
GiftId + Constituent + Amount + Status
```

Those may deserve:

EXAMPLE / FLOW

```
different constructors
or
factory methods
```

rather than a giant constructor with many optional parameters.

Constructor Parameter Explosion

Bad:

C#

```
public Gift(
    long giftId,
    long tenantId,
    long constituentId,
    long? campaignId,
    decimal totalAmount,
    string currencyCode,
    string? notes,
    DateTime createdAt,
    DateTime? refundedAt,
    bool cancelled,
    string status,
    string source,
    string externalReference,
```

```
    string receiptNumber)
{
}
```


This is difficult to:

```
EXAMPLE / FLOW
Read

Call correctly

Maintain

Test
```

It may indicate the type has too many responsibilities or construction needs a different design.

Possible Alternatives

Depending on context:

```
EXAMPLE / FLOW
Value objects

Factory methods

Builder pattern

Parameter object

Required properties

Separate creation/restoration paths

Smaller object responsibilities
```

Do not automatically replace every long constructor with a builder; investigate the design first.

Expression-Bodied Constructor

A short constructor can use:

```
EXAMPLE / FLOW
=>
```

Example:

```
C#
public Gift(
    decimal amount)
    =>
    TotalAmount = amount;
```

Equivalent conceptually to:

```
C#
public Gift(
    decimal amount)
{
```

```
    TotalAmount = amount;
}
```


Use it only when the constructor remains genuinely simple.

Field Initializers

Fields can be initialized at declaration.

Example:

```C#
public sealed class Gift
{
    private readonly List<long>
        _fundIds = [];
}
```

Another example:

```C#
private DateTime _createdAtUtc = DateTime.UtcNow;
```

Field initializers run as part of instance initialization before the constructor body for the object is completed.

Property Initializers

Auto-properties can also have initial values.

Example:

```C#
public string CurrencyCode
{
    get;
    private set;
} = "USD";
```

Another:

```C#
public List<long> FundIds
{
    get;
} = [];
```

Constructor Can Override an Initializer

Example:

```C#
public sealed class Gift
{
    public string CurrencyCode
    {
        get;
        private set;
    } = "USD";
```

```
    public Gift(
        string currencyCode)
    {
        CurrencyCode = currencyCode;
    }
}
```

If caller writes:

```C#
new Gift(
    "EUR");
```

final value becomes:

```EXAMPLE / FLOW
EUR
```

because constructor assignment occurs after the initializer for that instance. Microsoft notes that a constructor assignment can overwrite a value supplied by a field initializer.

Good Use of Field Initializers

Use field/property initializers for natural defaults.

Example:

```C#
private readonly List<GiftAllocation>
    _allocations = [];
```

This communicates:

```EXAMPLE / FLOW
Every new Gift begins
with an empty allocation collection.
```

No constructor needs to repeat:

```C#
_allocations = new List<GiftAllocation>();
```

Constructor vs Field Initializer

Use a field/property initializer when:

```EXAMPLE / FLOW
default does not depend
on constructor arguments
```

Example:

```C#
private readonly List<GiftAllocation>
    _allocations = [];
```

Use constructor initialization when:

EXAMPLE / FLOW

```
value depends on caller input
```

Example:

C#

```
TenantId = tenantId;
```

Object Initializers

Object initializer syntax lets callers assign accessible fields or properties after the constructor runs as part of the creation expression.

Example:

C#

```
Campaign campaign =
    new Campaign
    {
        Name = "Annual Giving",

        GoalAmount =
            500_000m
    };
```

Object initializers execute after constructor execution, in textual order.

Object Initializer Flow

EXAMPLE / FLOW

```
new Campaign
    →
Constructor
    →
Name assignment
    →
GoalAmount assignment
    →
Object returned
```

Object Initializers Do Not Replace Constructors Completely

Object initializers are convenient for:

EXAMPLE / FLOW

```
Optional values

DTO-style objects

Configuration-like models
```

But constructors remain useful for:

EXAMPLE / FLOW

```
Required state

Validation
```

Constructor + Object Initializer

Example:

C#

```
public sealed class Gift
{
    public Gift(
        long constituentId,
        decimal amount)
    {
        ConstituentId = constituentId;

        TotalAmount = amount;
    }

    public long ConstituentId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public string? Notes
    {
        get;
        init;
    }
}
```

Usage:

C#

```
Gift gift =
    new Gift(
        constituentId: 5001,
        amount: 1000m)
    {
        CampaignId = 2001,

        Notes =
            "Annual appeal"
    };
```

Conceptually:

EXAMPLE / FLOW

```
Required Core State
    →
```

```
Constructor
    *
Optional Initial State
    →
Object Initializer
```

`init` Properties

An `init` accessor permits assignment during object initialization, but prevents ordinary later assignment.

Example:

```C#
public string? Notes
{
    get;
    init;
}
```

Allowed:

```C#
Gift gift = new Gift(...)
    {
        Notes =
            "Annual appeal"
    };
```

Later:

```C#
// gift.Notes = //     "Changed";
```

is not allowed by ordinary caller code.

Microsoft documents `init` as assignment restricted to construction/initialization contexts.

Constructor vs `init`

Constructor:

```EXAMPLE / FLOW
forces value
through constructor call
```

`init`:

```EXAMPLE / FLOW
allows caller to provide value
through object initializer
```

Example constructor:

```C#
public Gift(
    long tenantId)
{
    TenantId = tenantId;
}
```

Example `init`:

C#
```
public string? Notes
{
    get;
    init;
}
```

A useful design might be:

EXAMPLE / FLOW
```
TenantId
    →
required invariant
    →
constructor

Notes
    →
optional initialization
    →
init
```

Get-Only Auto-Properties

Example:

C#
```
public long TenantId
{
    get;
}
```

A get-only auto-property can be assigned from constructors.

C#
```
public Gift(
    long tenantId)
{
    TenantId = tenantId;
}
```

After construction, outside callers can't assign it.

Microsoft specifically documents get-only auto-properties as assignable from constructors.

`required` Members

C# provides:

EXAMPLE / FLOW
```
required
```

to say:

> ⓘ The caller must initialize this member as part of object creation unless a constructor contract tells the compiler it has already done so.

Example:

C#
```
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }

    public decimal GoalAmount
    {
        get;
        init;
    }
}
```

Valid:

C#
```
Campaign campaign =
    new Campaign
    {
        Name =
            "Annual Giving"
    };
```

Invalid:

C#
```
// Campaign campaign = //     new Campaign();
```

because:

EXAMPLE / FLOW
```
Name
```

was not initialized.

`required` Is Not Constructor Parameter Syntax

These represent two different approaches.

Constructor:

C#
```
public Campaign(
    string name)
{
    Name = name;
}
```

Required property:

C#
```
public required string Name
{
    get;
```

```
    init;
}
```

Both can ensure callers provide data, but the call site differs.

Constructor Style

C#

```
Campaign campaign =
    new Campaign(
        "Annual Giving");
```

Required-Property Style

C#

```
Campaign campaign =
    new Campaign
    {
        Name =
            "Annual Giving"
    };
```

Constructor vs required

Constructor often works well when:

EXAMPLE / FLOW

```
A small number of values
are fundamental invariants
```

`required` properties can work well when:

EXAMPLE / FLOW

```
Many named initialization values
are needed
and named call-site syntax improves clarity
```

required Does Not Mean Non-Null

Important:

C#

```
public required string Name
{
    get;
    init;
}
```

requires initialization.

But callers could still attempt:

C#

```
Name =
    null
```

The nullable compiler analysis warns because `string` is non-nullable, but `required` itself means initialization obligation rather than automatic validation. Microsoft exp

CONSTRUCTORS / INITIALIZATION

`required` with non-nullability.

`required` Does Not Mean Business Valid

This satisfies initialization:

```
C#
Name = "";
```

but may violate your business rule.

Therefore:

```
EXAMPLE / FLOW
required
    ≠
validation
```

Constructors and Required Members

Suppose:

```
C#
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }

    public Campaign(
        string name)
    {
        Name = name;
    }
}
```

You might expect:

```
C#
new Campaign(
    "Annual Giving");
```

to automatically satisfy the compiler's required-member contract.

But simply assigning it inside an ordinary constructor isn't enough for the compiler to assume that constructor fulfills all required members.

That's where:

```
EXAMPLE / FLOW
SetsRequiredMembers
```

can be used.

SetsRequiredMembers

Namespace:

```
C#
System.Diagnostics.CodeAnalysis
```

Example:

```
C#
using System.Diagnostics.CodeAnalysis;

public sealed class Campaign
{
    [SetsRequiredMembers]
    public Campaign(
        string name)
    {
        Name = name;
    }

    public required string Name
    {
        get;
        init;
    }
}
```

Now:

```
C#
Campaign campaign =
    new Campaign(
        "Annual Giving");
```

doesn't also require:

```
C#
{
    Name = ...
}
```

because the constructor declares that it satisfies required members.

Important SetsRequiredMembers Rule

The attribute tells the compiler:

```
EXAMPLE / FLOW
Trust this constructor
to initialize required members.
```

It does not magically perform the assignments.

If you incorrectly apply it:

```
EXAMPLE / FLOW
your constructor can still
leave bad state
```

So use it only when the constructor actually establishes the required-member contract.

Constructor Initialization Order

This is a very important interview topic.

For a class instance created with `new`, the current Microsoft documentation summarizes initialization in this order:

EXAMPLE / FLOW

```
1. Instance fields receive default/zero values

2. Most-derived type's field initializers run

3. Base-type field initializers run
   through the inheritance chain

4. Base instance constructors run
   from Object toward the direct base

5. Current type's instance constructor body runs

6. Object initializer assignments run
```

A static constructor, if present, runs before the instance-construction actions for that type and runs at most once.

This order can feel unintuitive, so understand it rather than memorizing only one line.

Simplified Initialization Flow

EXAMPLE / FLOW

```
Defaults
    →
Field / Property Initializers
    →
Base Constructor Chain
    →
Current Constructor Body
    →
Object Initializer
    →
Object Ready
```

Example Initialization Order

C#

```
public class Gift
{
    private string _status =
        PrintAndReturn(
            "Field initializer");

    public Gift()
    {
        Console.WriteLine(
            "Constructor body");
    }

    public string Notes
```

```
    {
        get;
        set;
    } =
        PrintAndReturn(
            "Property initializer");

    private static string PrintAndReturn(
        string text)
    {
        Console.WriteLine(
            text);

        return text;
    }
}
```

Then:

C#

```
Gift gift =
    new Gift
    {
        Notes =
            "Object initializer"
    };
```

Conceptually:

EXAMPLE / FLOW

```
instance default initialization
    →
field/property initializers
    →
constructor body
    →
object initializer
```

Object Initializer Can Overwrite Constructor Values

Example:

C#

```
public sealed class Gift
{
    public Gift()
    {
        Notes = "Constructor";
    }

    public string Notes
    {
        get;
        set;
    }
}
```

Call:

```
C#
Gift gift =
    new Gift
    {
        Notes =
            "Object Initializer"
    };
```

Final:

```
EXAMPLE / FLOW
Notes
=
Object Initializer
```

because object initializer assignments run after the constructor.

Why This Matters for Invariants

Suppose:

```
C#
public Gift()
{
    Amount = 100m;
}

public decimal Amount
{
    get;
    set;
}
```

Caller can still do:

```
C#
new Gift
{
    Amount =
        -500m
};
```

So constructor validation alone cannot protect state if you expose unrestricted public setters afterward.

This connects directly to Topic 11:

```
EXAMPLE / FLOW
Constructor
+
Encapsulation
```

must work together.

Better

```
C#
public decimal Amount
{
    get;
```

```
    private set;
}
```

or:

```C#
public decimal Amount
{
    get;
    init;
}
```

with validation appropriate to the design.

Static Constructors

Definition

A static constructor initializes type-level static state.

Syntax:

```C#
public sealed class GiftRules
{
    static GiftRules()
    {
    }
}
```

Notice:

```EXAMPLE / FLOW
No access modifier

No parameters

No explicit return type
```

Static constructors are invoked automatically by the runtime, run at most once, and occur before instance construction/use requiring type initialization.

Static Constructor Example

```C#
public static class ApplicationMetadata
{
    public static readonly DateTime
        StartedAtUtc;

    static ApplicationMetadata()
    {
        StartedAtUtc = DateTime.UtcNow;
    }
}
```

Static Initialization Flow

EXAMPLE / FLOW

```
Type first needs initialization
    →
Static fields receive defaults
    →
Static field initializers run
    →
Static constructor runs
    →
Type ready
```

Static field initializers run in textual order before the static constructor.

Static Constructor Rules

A static constructor:

EXAMPLE / FLOW

```
Cannot have parameters

Cannot have access modifiers

Cannot be explicitly called

Cannot use : this(...)

Cannot use : base(...)

Runs automatically

Runs at most once
```

These restrictions are documented by the current compiler rules.

Static Constructor Exception

Suppose:

C#

```
static GiftRules()
{
    throw new Exception(
        "Initialization failed");
}
```

If the static constructor fails, the type can remain uninitialized for the lifetime of that runtime context; subsequent access commonly results in `TypeInitializationException`. The runtime doesn't simply rerun it on every attempt.

Therefore static constructors should avoid fragile external work.

Bad Static Constructor

Avoid:

```C#
static Configuration()
{
    Settings = LoadFromRemoteServer();

    Database = ConnectToDatabase();
}
```

Problems:

EXAMPLE / FLOW

```
Network failure

Database unavailable

Blocking startup

Type initialization failure

Hard testing
```

Use normal dependency-injection/configuration infrastructure instead.

Prefer Inline Static Initialization When Simple

Instead of:

```C#
public static readonly TimeSpan
    Timeout;

static GiftRules()
{
    Timeout =
        TimeSpan.FromSeconds(
            30);
}
```

prefer:

```C#
public static readonly TimeSpan
    Timeout =
        TimeSpan.FromSeconds(
            30);
```

when no additional static-construction logic is necessary.

Private Constructors

Private constructors are not only for singletons.

Example utility/factory-style design:

```C#
public sealed class GiftReference
{
    private GiftReference(
        string value)
    {
```

```
        Value = value;
    }

    public string Value
    {
        get;
    }

    public static GiftReference Create(
        long giftId)
    {
        return new GiftReference(
            $"GFT-{giftId}");
    }
}
```

This guarantees callers go through:

```
Create(...)
```

rather than arbitrary construction.

Primary Constructors

Modern C# introduced primary constructors for classes and structs in:

```
C# 12
```

and they remain available in current C#.

Syntax:

```
public class Gift(
    long constituentId,
    decimal amount)
{
}
```

Parameters appear directly after the type name.

Primary Constructor Example

```
public sealed class Gift(
    long constituentId,
    decimal amount)
{
    public long ConstituentId
    {
        get;
    } = constituentId;

    public decimal TotalAmount
    {
        get;
        private set;
```

```
    } = amount;
}
```


Usage:

```C#
Gift gift =
    new Gift(
        5001,
        1000m);
```

Primary Constructor Parameters Are Parameters

This is extremely important.

Given:

```C#
public class Gift(
    long constituentId)
{
}
```

`constituentId` is **not automatically a property or field**.

It is a constructor parameter whose scope extends throughout the type body.

This Does Not Automatically Create

```C#
public long ConstituentId
{
    get;
}
```

for a normal class.

Records are different.

Primary constructor parameters in normal classes and structs do not automatically become public properties.

Wrong Assumption

```C#
public class Gift(
    long constituentId)
{
}
```

does **not** mean callers can use:

```C#
gift.constituentId
```

The parameter isn't a normal class member and can't be accessed as:

either.

Capture into a Property

```C#
public sealed class Gift(
    long constituentId,
    decimal amount)
{
    public long ConstituentId
    {
        get;
    } = constituentId;

    public decimal TotalAmount
    {
        get;
    } = amount;
}
```

Now the object exposes proper properties.

Primary Constructor Parameter Storage

Interesting advanced rule:

EXAMPLE / FLOW

```
Primary constructor parameter
```

does not automatically require a backing field.

The compiler creates storage only when needed.

If a parameter is used only to initialize a field/property:

```C#
public class Gift(
    long id)
{
    public long GiftId
    {
        get;
    } = id;
}
```

the parameter itself may not require separate long-term storage.

If the parameter is referenced later from an instance member:

```C#
public class Gift(
    long id)
{
    public long GetId()
        =>
        id;
}
```

the compiler needs storage so that parameter value remains available. Microsoft documents this distinction explicitly.

Primary Constructor and Other Constructors

If a class declares a primary constructor, any other explicitly written instance constructors must call the primary constructor directly or indirectly using:

EXAMPLE / FLOW

```
this(...)
```

This ensures primary-constructor parameters are initialized for every creation path.

Example

C#

```
public sealed class Gift(
    long constituentId,
    decimal amount)
{
    public Gift(
        long constituentId)
        : this(
            constituentId,
            1m)
    {
    }
}
```

Valid because secondary constructor eventually invokes the primary constructor.

Primary Constructor Removes Implicit Class Parameterless Constructor

For a class with:

C#

```
public class Gift(
    long id)
{
}
```

the compiler does not also generate:

C#

```
Gift()
```

because the primary constructor defines required construction parameters.

Primary Constructors and Dependency Injection

A common modern usage is dependency injection.

Traditional:

C#

```
public sealed class GiftService
{
```

```
    private readonly IGiftRepository
        _giftRepository;

    private readonly ILogger<GiftService>
        _logger;

    public GiftService(
        IGiftRepository giftRepository,
        ILogger<GiftService> logger)
    {
        _giftRepository = giftRepository;

        _logger = logger;
    }
}
```

Primary-constructor version:

C#

```
public sealed class GiftService(
    IGiftRepository giftRepository,
    ILogger<GiftService> logger)
{
    public void CreateGift(
        Gift gift)
    {
        logger.LogInformation(
            "Creating gift {GiftId}",
            gift.GiftId);

        giftRepository.Save(
            gift);
    }
}
```

Microsoft specifically identifies dependency injection as a common primary-constructor use case.

Should Every Service Use Primary Constructors?

No.

Primary constructors are a language feature, not a requirement.

Traditional constructor syntax can be clearer when you need:

EXAMPLE / FLOW

```
Validation

Parameter transformation

Explicit readonly fields

Multiple initialization steps

Detailed debugging

More visible constructor logic
```

Choose the style that makes the class easiest to understand.

Traditional Constructor vs Primary Constructor

Traditional:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    public GiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Primary:

```C#
public sealed class GiftService(
    IGiftRepository repository)
{
}
```

Both can be valid.

Primary Constructor Caution: Duplicate Storage

Suppose:

```C#
public class Gift(
    long giftId)
{
    private readonly long _giftId = giftId;

    public long GetId()
        =>
        giftId;
}
```

Now the type might need storage for:

EXAMPLE / FLOW

```
_giftId
```

and separately the captured primary-constructor parameter:

EXAMPLE / FLOW

```
giftId
```

This can create unnecessary duplicated storage.

Microsoft warns about similar scenarios, especially across inheritance hierarchies.

Better:

```C#
public class Gift(
    long giftId)
{
    public long GiftId
    {
        get;
    } = giftId;

    public long GetId()
        =>
        GiftId;
}
```

Use the actual property after initialization.

Constructor and readonly

A classic use of constructors is assigning readonly fields.

Example:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    public GiftService(
        IGiftRepository repository)
    {
        ArgumentNullException.ThrowIfNull(
            repository);

        _repository = repository;
    }
}
```

Once construction completes:

```EXAMPLE / FLOW
_repository
```

cannot normally be reassigned.

This gives the object a stable dependency.

Constructor Dependency Injection

ASP.NET Core commonly uses constructor injection.

Concept:

```EXAMPLE / FLOW
DI Container
    →
find constructor
    →
resolve dependencies
    →
call constructor
```

```
    →
return service
```


Example:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository
        _repository;

    private readonly ILogger<GiftService>
        _logger;

    public GiftService(
        IGiftRepository repository,
        ILogger<GiftService> logger)
    {
        _repository = repository;

        _logger = logger;
    }
}
```

The service explicitly communicates:

```EXAMPLE / FLOW
To construct GiftService,
I require:

IGiftRepository
ILogger<GiftService>
```

This is much better than hiding dependencies in static globals.

Dependency Constructor as Contract

Constructor:

```C#
public GiftService(
    IGiftRepository repository,
    ILogger<GiftService> logger)
```

acts as a declaration of:

```EXAMPLE / FLOW
Required Dependencies
```

The object cannot normally exist without them.

Avoid Service Locator in Constructor

Bad:

```C#
public GiftService()
{
    _repository = GlobalServices.Get<IGiftRepository>();
```

```
    _logger = GlobalServices.Get<ILogger<GiftService>>();
}
```


Dependencies are now hidden.

Better:

```C#
public GiftService(
    IGiftRepository repository,
    ILogger<GiftService> logger)
```

Dependencies become explicit and testable.

Constructor Should Not Do Heavy I/O

Avoid:

```C#
public GiftService()
{
    _gifts = LoadAllGiftsFromDatabase();

    _token = CallAuthenticationServer();

    _configuration = DownloadRemoteConfiguration();
}
```

Problems:

EXAMPLE / FLOW

```
Construction becomes slow

Failure-prone

Hard to test

Hard to retry

Unexpected side effects

DI resolution may block
```

Constructors should generally initialize state, validate dependencies, and establish invariants—not perform large workflows.

Constructor Side Effects

Avoid significant side effects such as:

EXAMPLE / FLOW

```
Sending email

Writing database records

Publishing messages

Calling APIs

Starting background jobs
```

```
Deleting files
```

inside constructors.

Why?

A caller expects:

EXAMPLE / FLOW

```
new SomeType(...)
```

to construct an object.

Not:

EXAMPLE / FLOW

```
perform half the business workflow
```

Better

Construct object:

C#

```
Gift gift = new Gift(...);
```

Then explicitly:

C#

```
await paymentService.ProcessAsync(
    gift);
```

or:

C#

```
await giftRepository.SaveAsync(
    gift);
```

The intent is visible.

Do Not Call Virtual Members from Constructors

This becomes more important in inheritance.

A constructor calling an overridable virtual method can run derived behavior **before the derived type has finished initializing**.

That can observe partially initialized state.

Avoid designs such as:

C#

```
public BaseClass()
{
    Initialize();
}

protected virtual void Initialize()
{
}
```

We'll revisit this in inheritance and polymorphism.

Constructor Exceptions

A constructor may throw an exception when initialization fails.

Example:

C#
```
public Gift(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    Amount = amount;
}
```

Call:

C#
```
Gift gift =
    new Gift(
        -100m);
```

fails.

The caller does not receive a successfully constructed valid `Gift`.

Good Constructor Exceptions

Use specific exceptions where appropriate:

EXAMPLE / FLOW
```
ArgumentNullException

ArgumentException

ArgumentOutOfRangeException

InvalidOperationException
```

depending on the situation.

For invalid constructor argument:

EXAMPLE / FLOW
```
Argument...
```

exceptions are usually more precise than:

C#
```
throw new Exception();
```

Avoid Catching Your Own Validation Exception Inside Constructor

Bad:

```C#
public Gift(
    decimal amount)
{
    try
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException();
        }
    }
    catch
    {
        Amount = 0m;
    }
}
```

Now invalid input silently creates:

```EXAMPLE / FLOW
Amount = 0
```

which may violate your model.

If input invalidates construction:

```EXAMPLE / FLOW
fail construction
```

rather than hide the error.

Constructor Null Validation

Good:

```C#
public GiftService(
    IGiftRepository repository)
{
    ArgumentNullException.ThrowIfNull(
        repository);

    _repository = repository;
}
```

Even with nullable reference types:

```EXAMPLE / FLOW
runtime validation
```

can still matter at boundaries.

Constructor Parameter Assignment Pattern

Classic:

```C#
private readonly IGiftRepository
    _giftRepository;

public GiftService(
    IGiftRepository giftRepository)
{
    _giftRepository = giftRepository;
}
```

Field:

EXAMPLE / FLOW

```
_giftRepository
```

Parameter:

EXAMPLE / FLOW

```
giftRepository
```

This naming style makes assignment clear.

`this` to Disambiguate

Alternative:

```C#
private readonly IGiftRepository
    giftRepository;

public GiftService(
    IGiftRepository giftRepository)
{
    this.giftRepository = giftRepository;
}
```

Both work.

The `_fieldName` convention generally avoids needing `this` for simple assignments.

Constructors and Object Identity

Suppose:

```C#
Gift first =
    new Gift(
        1000m);

Gift second =
    new Gift(
        1000m);
```

The same constructor values do not mean:

EXAMPLE / FLOW

```
same object
```

There are:

EXAMPLE / FLOW

```
Gift Object A
+
Gift Object B
```

Classes have reference identity.

Constructor Is Not Object Reuse

Every:

C#

```
new Gift(...)
```

normally means:

EXAMPLE / FLOW

```
new instance
```

unless you're using a special factory/pooling framework instead of direct construction.

Constructors and Mutable Objects

A constructor establishes initial state.

It doesn't automatically make the object immutable.

Example:

C#

```
public Gift(
    decimal amount)
{
    Amount = amount;
}

public decimal Amount
{
    get;
    set;
}
```

After construction:

C#

```
gift.Amount = -100m;
```

is still possible.

Therefore:

EXAMPLE / FLOW

```
Constructor validation
+
Encapsulation
```

must be designed together.

Constructors and Collections

Example:

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public IReadOnlyList<GiftAllocation>
        Allocations
        =>
        _allocations;
}
```

The list exists immediately for every instance.

This avoids:

EXAMPLE / FLOW

```
null collection
```

and makes later methods simpler.

Bad Collection Constructor Pattern

C#

```
private List<GiftAllocation>?
    _allocations;
```

then everywhere:

C#

```
_allocations ??= [];
```

If an empty collection is a valid natural state, initialize it once.

Defensive Copying in Constructors

Suppose caller provides an array:

C#

```
public Batch(
    long[] giftIds)
{
    _giftIds = giftIds;
}
```

Problem:

```C#
long[] source =
[
    1001,
    1002
];

Batch batch =
    new Batch(
        source);

source[0] = 9999;
```

If `Batch` stored the same array reference, caller mutated its internal state indirectly.

Defensive Copy

```C#
public Batch(
    long[] giftIds)
{
    ArgumentNullException.ThrowIfNull(
        giftIds);

    _giftIds = giftIds[..];
}
```

Now:

EXAMPLE / FLOW

```
caller array
and
internal array
```

are different array objects.

This is an example of constructor-based encapsulation.

Constructor and Date/Time

Avoid hiding unpredictable date/time acquisition when testability matters.

Instead of:

```C#
public Gift()
{
    CreatedAtUtc = DateTime.UtcNow;
}
```

sometimes infrastructure/domain designs benefit from passing time explicitly:

```C#
public Gift(
    DateTime createdAtUtc)
{
    CreatedAtUtc = createdAtUtc;
}
```

or injecting a clock/time provider in a service.

The correct design depends on how deterministic/testable the object needs to be.

Struct Constructors — Important Distinction

Although this topic mainly focuses on classes, structs can also define constructors.

Example:

```C#
public readonly struct Money
{
    public Money(
        decimal amount,
        string currencyCode)
    {
        Amount = amount;

        CurrencyCode = currencyCode;
    }

    public decimal Amount
    {
        get;
    }

    public string CurrencyCode
    {
        get;
    }
}
```

new Struct() vs default

Modern C# structs can declare parameterless constructors.

Important distinction:

```C#
var custom = new SomeStruct();
```

can run the explicit parameterless constructor.

But:

```C#
var value = default(SomeStruct);
```

produces the struct's default zero-initialized representation instead of invoking custom constructor logic. Microsoft documents this distinction explicitly.

Structs will be covered properly in Topic 17.

Primary Constructors and Records

Primary constructors existed for records before general classes/structs received them.

For a record:

```C#
public record GiftDto(
    long GiftId,
    decimal Amount);
```

the positional parameters become record properties.

For a normal class:

```C#
public class Gift(
    long giftId,
    decimal amount)
{
}
```

they do **not** automatically become properties.

This distinction is frequently asked in interviews.

C# 14: Partial Constructors

Current C# 14 adds support for **partial instance constructors**.

A partial constructor has:

```EXAMPLE / FLOW
one defining declaration
+
one implementing declaration
```

Only the implementing declaration can contain a constructor initializer such as:

```EXAMPLE / FLOW
this(...)
or
base(...)
```

This is a newer language feature and is mainly useful in scenarios involving source generation and partial types rather than ordinary application classes.

Partial Constructor Example — Conceptual

```C#
public partial class GeneratedGift
{
    public partial GeneratedGift(
        long id);
}
```

Another partial declaration provides implementation:

```C#
public partial class GeneratedGift
{
    public partial GeneratedGift(
        long id)
    {
        GiftId = id;
    }
```

```
    public long GiftId
    {
        get;
    }
}
```

For everyday application code:

EXAMPLE / FLOW

```
normal constructor
```

remains the standard.

Do not introduce partial constructors unless the code-generation/partial-type scenario benefits from them.

Constructor Design Strategy

When designing a constructor, ask:

EXAMPLE / FLOW

```
1. What state is mandatory?

2. What state is optional?

3. Which values must never change?

4. Which values require validation?

5. Which dependencies are required?

6. Can this object exist meaningfully
   without each parameter?

7. Does the constructor have too many parameters?

8. Am I doing expensive I/O
   during construction?

9. Can caller put the object
   into invalid state afterward?

10. Should creation use a factory
    instead?
```

Required vs Optional State

Example `Gift`:

Required:

EXAMPLE / FLOW

```
TenantId

ConstituentId

Amount

Currency
```

Optional:

EXAMPLE / FLOW

```
CampaignId

Notes
```

One reasonable design:

C#

```
public Gift(
    long tenantId,
    long constituentId,
    decimal amount,
    string currencyCode)
{
    ...
}

public long? CampaignId
{
    get;
    init;
}

public string? Notes
{
    get;
    init;
}
```

Constructor Call

C#

```
Gift gift =
    new Gift(
        tenantId: 1,
        constituentId: 5001,
        amount: 1000m,
        currencyCode: "USD")
    {
        CampaignId = 2001,

        Notes =
            "Annual campaign"
    };
```

This separates:

EXAMPLE / FLOW

```
required invariant state
```

from:

EXAMPLE / FLOW

```
optional initialization data
```

When Required Properties May Be Better

Suppose an API DTO has six required values:

```C#
public sealed class CreateGiftRequest
{
    public required long ConstituentId
    {
        get;
        init;
    }

    public required decimal Amount
    {
        get;
        init;
    }

    public required string CurrencyCode
    {
        get;
        init;
    }

    public required DateOnly GiftDate
    {
        get;
        init;
    }
}
```

Call:

```C#
CreateGiftRequest request =
    new CreateGiftRequest
    {
        ConstituentId = 5001,

        Amount = 1000m,

        CurrencyCode = "USD",

        GiftDate =
            DateOnly.FromDateTime(
                DateTime.UtcNow)
    };
```

Named initializer syntax can be easier to read than:

```C#
new CreateGiftRequest(
    5001,
    1000m,
    "USD",
    someDate);
```

especially for DTOs.

Domain Entity vs DTO Construction

DTO:

EXAMPLE / FLOW

```
Data transfer
    →
required/init properties often convenient
```

Domain entity:

EXAMPLE / FLOW

```
Business invariant
    →
constructor/factory often stronger
```

This is not a hard rule, but a useful design tendency.

ASP.NET Core Controller Construction

Controllers and services are normally constructed by:

EXAMPLE / FLOW

```
ASP.NET Core DI container
```

Example:

C#

```
public sealed class GiftsController(
    IGiftService giftService,
    ILogger<GiftsController> logger)
    : ControllerBase
{
}
```

or traditional:

C#

```
public sealed class GiftsController :
    ControllerBase
{
    private readonly IGiftService
        _giftService;

    private readonly ILogger<GiftsController>
        _logger;

    public GiftsController(
        IGiftService giftService,
        ILogger<GiftsController> logger)
    {
        _giftService = giftService;

        _logger = logger;
    }
}
```

The framework resolves constructor dependencies.

Constructor Injection Advantages

Constructor injection makes dependencies:

EXAMPLE / FLOW

```
Explicit

Mandatory

Testable

Visible

Stable
```

A test can provide:

EXAMPLE / FLOW

```
Fake Repository

Mock Logger

Stub Service
```

when creating the object.

Constructor Injection Test Example

C#

```
var fakeRepository = new FakeGiftRepository();

var service =
    new GiftService(
        fakeRepository);
```

No global service locator is necessary.

Too Many Injected Dependencies

If a service constructor becomes:

C#

```
public GiftService(
    IRepositoryA a,
    IRepositoryB b,
    IServiceC c,
    IServiceD d,
    IServiceE e,
    IServiceF f,
    IServiceG g,
    IServiceH h)
```

do not automatically hide the problem using:

EXAMPLE / FLOW

```
ServiceProvider
```

or a dependency bag.

A long constructor can be a design smell indicating:

EXAMPLE / FLOW

```
too many responsibilities
```

This becomes important in SOLID/SRP later.

Constructor and Captive Dependency Preview

ASP.NET Core service lifetimes matter.

Example:

EXAMPLE / FLOW

```
Singleton service
    →
constructor receives scoped service
```

can create a captive-lifetime problem.

We will study that deeply under Dependency Injection and Service Lifetimes.

For now remember:

EXAMPLE / FLOW

```
Constructor injection
doesn't remove the need
to respect dependency lifetimes.
```

Security Considerations

Constructor design can protect security-sensitive invariants.

Example:

C#

```
public UserContext(
    long tenantId,
    long userId)
{
    if (tenantId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(tenantId));
    }

    if (userId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(userId));
    }

    TenantId = tenantId;

    UserId = userId;
}
```

Then expose:

C#

```
public long TenantId
{
    get;
}
```

instead of:

```C#
public long TenantId
{
    get;
    set;
}
```

which could allow arbitrary tenant reassignment.

Never Accept Sensitive State Blindly

Example:

```C#
public User(
    bool isAdministrator)
{
    IsAdministrator = isAdministrator;
}
```

may be dangerous if caller input can freely control it.

Security-sensitive state should originate from trusted authorization processes.

A constructor doesn't automatically make a value trustworthy.

Validate External Constructor Input

If constructors receive data from:

```EXAMPLE / FLOW
HTTP request

Database

File

External API

Deserialization
```

do not assume it is safe merely because a constructor is being used.

Validate according to domain requirements.

Performance Considerations

Constructors themselves are ordinary initialization code.

One constructor assignment such as:

```C#
_id = id;
```

is not a meaningful performance concern.

Focus on:

```EXAMPLE / FLOW
What the constructor does
```

rather than constructor syntax.

Expensive Constructor Problem

This:

```C#
public GiftService()
{
    _gifts = database.LoadOneMillionGifts();
}
```

is expensive because of:

```EXAMPLE / FLOW
database work
+
allocations
```

not because it is a constructor.

Object Allocation Cost

Creating many class instances means:

```EXAMPLE / FLOW
managed allocations
```

and eventually:

```EXAMPLE / FLOW
garbage collection
```

But don't avoid constructors simply to avoid allocations.

Choose class vs struct based on semantics first.

Primary Constructor Performance

Primary constructors don't inherently make a class faster just because syntax is shorter.

Their main benefit is:

```EXAMPLE / FLOW
conciseness
+
clear required constructor inputs
```

Compiler storage depends on how primary-constructor parameters are used.

Avoid Unnecessary Duplicate Storage

Poor:

```C#
public sealed class Gift(
    long giftId)
{
    private readonly long _giftId = giftId;
```

```
    public long GetGiftId()
        =>
        giftId;
}
```

Potentially stores:

EXAMPLE / FLOW

```
field
+
captured primary constructor parameter
```

Better:

C#

```
public sealed class Gift(
    long giftId)
{
    public long GiftId
    {
        get;
    } = giftId;

    public long GetGiftId()
        =>
        GiftId;
}
```

Common Mistake — Adding Empty Constructor "Just in Case"

Bad:

C#

```
public Gift()
{
}
```

if every valid gift requires:

EXAMPLE / FLOW

```
ConstituentId
+
Amount
```

This allows invalid instances.

Common Mistake — Forgetting Compiler Constructor Rule

Given:

C#

```
public Gift(
    decimal amount)
{
}
```

this no longer exists automatically:

```C#
Gift()
```

If you need it, declare it.

Common Mistake — Duplicating Constructor Logic

Bad:

```EXAMPLE / FLOW
Constructor A
    →
validate amount

Constructor B
    →
validate amount again

Constructor C
    →
validate amount again
```

Prefer:

```EXAMPLE / FLOW
this(...)
    →
central constructor
```

when overloads truly represent variations of the same initialization path.

Common Mistake — Circular this Chaining

Never:

```EXAMPLE / FLOW
Gift()
    →
Gift(decimal)
    →
Gift()
```

The compiler rejects constructor cycles.

Common Mistake — Forgetting Base Constructor Requirement

If base class has only:

```C#
Base(
    int value)
```

then derived class cannot rely on implicit:

```C#
base()
```

It must invoke a valid base constructor.

Common Mistake — Constructor Does Everything

Bad:

EXAMPLE / FLOW

```
Constructor
    →
Load database
    →
Call HTTP API
    →
Send email
    →
Write file
    →
Publish event
```

Constructors should not become workflows.

Common Mistake — Public Setters Undo Constructor Validation

Constructor:

C#

```
public Gift(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw ...
    }

    Amount = amount;
}
```

But:

C#

```
public decimal Amount
{
    get;
    set;
}
```

means caller can later do:

C#

```
gift.Amount = -500m;
```

Encapsulation must continue after construction.

Common Mistake — Object Initializer Bypasses Intended Invariant

If:

```C#
public decimal Amount
{
    get;
    set;
}
```

then:

```C#
new Gift
{
    Amount =
        -100m
};
```

can create invalid state.

Don't expose setters merely for initializer convenience.

Common Mistake — required = Validation

Wrong:

```
required string Name
```

does not mean:

```
non-empty valid business name
```

It means:

```
caller must initialize it
```

Common Mistake — SetsRequiredMembers Does Validation

No.

It tells the compiler:

```
constructor satisfies required members
```

It does not validate the constructor implementation.

Common Mistake — Primary Constructor Parameter Is Property

Wrong for normal classes:

```C#
public class Gift(
    long giftId)
```

does not automatically expose:

```C#
gift.GiftId
```

You must explicitly define a property if you want one.

Common Mistake — Using Primary Constructor Parameter and Property Separately

Example:

```C#
public class Gift(
    decimal amount)
{
    public decimal Amount
    {
        get;
    } = amount;

    public decimal Calculate()
        =>
        amount * 2m;
}
```

Now method uses:

```
EXAMPLE / FLOW
captured constructor parameter
```

while callers see:

```
EXAMPLE / FLOW
Amount property
```

If state later changes, these could diverge depending on design.

Prefer using the intended state member consistently.

Common Mistake — Static Constructor with I/O

Avoid network/database logic inside a static constructor.

Static-constructor failure can make the type unusable for the runtime lifetime.

Common Mistake — Treating Constructor Like a Normal Method

You cannot call:

```C#
gift.Gift();
```

to rerun its constructor.

Construction is tied to object creation/chaining.

If you need reset behavior:

EXAMPLE / FLOW

```
provide an explicit method
```

but consider carefully whether resetting is good object design.

Common Mistake — Returning Value from Constructor

Invalid:

C#

```
public Gift()
{
    // return someGift;
}
```

Constructors have no return type.

You can use:

EXAMPLE / FLOW

```
return;
```

to exit early in some situations, but you cannot return another object/value as the constructor's result.

Factories are appropriate when creation needs to return something conditionally or choose a subtype.

Common Mistake — Async Constructor

C# constructors cannot be declared:

C#

```
async
```

You cannot write:

C#

```
public async Gift()
```

If creation requires asynchronous work, common approaches include:

EXAMPLE / FLOW

```
Async factory method

Separate InitializeAsync

Service responsible for async loading
```

Example:

C#

```
public static async Task<GiftService>
    CreateAsync(...)
```

depending on design.

Why Async Constructors Are Difficult

Constructor contract is:

EXAMPLE / FLOW

```
new
    →
returns initialized instance
```

Async requires:

EXAMPLE / FLOW

```
return later through Task
```

These models don't fit ordinary constructor syntax.

Better Async Factory Example

C#

```
public sealed class GiftImporter
{
    private GiftImporter(
        string configuration)
    {
        Configuration = configuration;
    }

    public string Configuration
    {
        get;
    }

    public static async Task<GiftImporter>
        CreateAsync(
            IConfigurationClient client,
            CancellationToken cancellationToken)
    {
        string configuration =
            await client.GetConfigurationAsync(
                cancellationToken);

        return new GiftImporter(
            configuration);
    }
}
```

This makes asynchronous creation explicit.

Practical Exercise — Parameterless Constructor

C#

```
public sealed class Campaign
{
    public Campaign()
    {
        Name = "Unnamed Campaign";
    }

    public string Name
    {
        get;
        private set;
```

```
    }
}
```


Create:

```C#
Campaign campaign = new Campaign();
```

Result:

EXAMPLE / FLOW

```
Name
=
Unnamed Campaign
```

Practical Exercise — Parameterized Constructor

```C#
public sealed class Campaign
{
    public Campaign(
        string name)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(
            name);

        Name = name;
    }

    public string Name
    {
        get;
    }
}
```

Create:

```C#
Campaign campaign =
    new Campaign(
        "Annual Giving");
```

Practical Exercise — Overloaded Constructors

```C#
public sealed class Campaign
{
    public Campaign(
        string name)
        : this(
            name,
            goalAmount: 0m)
    {
    }

    public Campaign(
        string name,
        decimal goalAmount)
    {
```

```
        ArgumentException.ThrowIfNullOrWhiteSpace(
            name);

        if (goalAmount < 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(goalAmount));
        }

        Name = name;

        GoalAmount = goalAmount;
    }

    public string Name
    {
        get;
    }

    public decimal GoalAmount
    {
        get;
    }
}
```

Practical Exercise — Constructor Chaining

Trace:

C#

```
new Campaign(
    "Annual Giving");
```

Flow:

EXAMPLE / FLOW

```
Campaign(string)
    →
this(string, decimal)
    →
Campaign(string, decimal)
    →
validate
    →
assign properties
    →
return
```

Practical Exercise — Base Constructor

C#

```
public class Transaction
{
    public Transaction(
        decimal amount)
    {
        Amount = amount;
    }

    public decimal Amount
    {
```

```
        get;
    }
}

public sealed class Gift :
    Transaction
{
    public Gift(
        decimal amount)
        : base(
            amount)
    {
    }
}
```

Call:

```C#
new Gift(
    1000m);
```

Order:

```EXAMPLE / FLOW
Transaction constructor
    →
Gift constructor body
```

Practical Exercise — Private Constructor

```C#
public sealed class GiftReference
{
    private GiftReference(
        string value)
    {
        Value = value;
    }

    public string Value
    {
        get;
    }

    public static GiftReference Create(
        long giftId)
    {
        return new GiftReference(
            $"GFT-{giftId}");
    }
}
```

Outside:

```C#
GiftReference reference =
    GiftReference.Create(
        1001);
```

Cannot normally use:

```
C#
// new GiftReference(...)
```

Practical Exercise — required

```
C#
public sealed class Fund
{
    public required string Code
    {
        get;
        init;
    }

    public required string Name
    {
        get;
        init;
    }
}
```

Valid:

```
C#
Fund fund =
    new Fund
    {
        Code = "GENERAL",

        Name =
            "General Fund"
    };
```

Practical Exercise — Required Constructor

```
C#
using System.Diagnostics.CodeAnalysis;

public sealed class Fund
{
    [SetsRequiredMembers]
    public Fund(
        string code,
        string name)
    {
        Code = code;

        Name = name;
    }

    public required string Code
    {
        get;
        init;
    }

    public required string Name
    {
        get;
        init;
```

```
    }
}
```


Now:

```
C#
Fund fund =
    new Fund(
        "GENERAL",
        "General Fund");
```

satisfies required-member construction.

Practical Exercise — Primary Constructor

Traditional:

```
C#
public sealed class FundService
{
    private readonly IFundRepository
        _repository;

    public FundService(
        IFundRepository repository)
    {
        _repository = repository;
    }
}
```

Primary:

```
C#
public sealed class FundService(
    IFundRepository repository)
{
    public void Save(
        Fund fund)
    {
        repository.Save(
            fund);
    }
}
```

Both can be valid.

Practical Exercise — Defensive Copy

```
C#
public sealed class GiftBatch
{
    private readonly long[]
        _giftIds;

    public GiftBatch(
        long[] giftIds)
    {
        ArgumentNullException.ThrowIfNull(
            giftIds);
```

```
        _giftIds = giftIds[..];
    }

    public IReadOnlyList<long>
        GiftIds
        =>
        _giftIds;
}
```


The constructor prevents callers from changing the internal array by modifying their original array reference.

Interview Questions

What is a constructor in C#?

A constructor is a special member executed as part of creating an instance of a class or struct.

It initializes the object and can enforce valid initial state.

Does a constructor have a return type?

No.

Not even:

C#
```
void
```

If a member has a return type, it is not a constructor.

What must a constructor be named?

For ordinary declared constructors, its name matches the containing type.

Example:

C#
```
public Gift()
{
}
```

inside:

C#
```
class Gift
```

What is a parameterless constructor?

A constructor that takes no parameters.

Example:

C#
```
public Gift()
{
}
```

When does C# generate a parameterless constructor automatically?

For a normal non-static class with no declared instance constructors, the compiler provides a public parameterless constructor.

Once you declare an instance constructor, that implicit class constructor is no longer added.

What is a parameterized constructor?

A constructor that accepts one or more parameters.

Example:

```C#
public Gift(
    decimal amount)
{
}
```

It allows required values to be provided during construction.

Can a class have multiple constructors?

Yes.

Constructor overloading allows several constructors with different parameter lists.

What is constructor overloading?

Defining multiple constructors for the same type with different signatures.

Example:

```C#
Gift()

Gift(decimal amount)

Gift(decimal amount, string currency)
```

What is constructor chaining?

Having one constructor invoke another constructor to reuse initialization logic.

Within the same class:

```C#
: this(...)
```

Base class:

```C#
: base(...)
```

What does `this(...)` mean in a constructor declaration?

It invokes another instance constructor in the same class before the current constructor body.

Why use constructor chaining?

To centralize initialization and avoid duplicated validation or assignment logic.

Instead of:

EXAMPLE / FLOW

```
three constructors
×
three copies of rules
```

use:

EXAMPLE / FLOW

```
multiple entry constructors
    →
one central constructor
```

Can constructors call each other circularly?

No.

A constructor cannot call itself directly or through a circular chain. The compiler rejects such chains.

What does base(...) do?

It invokes an accessible constructor in the direct base class.

The base constructor executes before the derived constructor body.

What happens if a derived constructor doesn't explicitly call base(...)?

C# implicitly attempts to call:

C#

```
base()
```

If the base type has no accessible parameterless constructor, the derived constructor must explicitly invoke another base constructor.

What is a private constructor?

A constructor accessible only inside the containing type.

It can be used to restrict direct object creation and support factories or other controlled creation patterns.

Can constructors have access modifiers?

Yes.

Instance constructors can use accessibility such as:

EXAMPLE / FLOW

```
public

private

protected

internal

protected internal

private protected
```

depending on design.

What is a static constructor?

A special constructor that initializes static/type-level state.

It has:

EXAMPLE / FLOW

```
No parameters

No access modifier
```

and is invoked automatically by the runtime.

How many times does a static constructor run?

At most once for that type initialization in the runtime context.

Can a static constructor be called manually?

No.

The runtime controls its execution.

Can a static constructor accept parameters?

No.

Can a static constructor use `this(...)` or `base(...)`?

No.

What happens if a static constructor throws?

The type initialization fails and the runtime doesn't simply retry the static constructor on each future access. `TypeInitializationException` commonly surfaces.

What is an object initializer?

Syntax that assigns accessible fields/properties as part of an object creation expression.

Example:

C#

```
new Campaign
{
    Name =
        "Annual Giving"
};
```

The constructor runs first, then object-initializer assignments run in textual order.

What is the difference between constructor initialization and object initializer?

Constructor:

EXAMPLE / FLOW

```
Type controls initialization logic
```

Object initializer:

EXAMPLE / FLOW

```
Caller assigns accessible members
after constructor execution
```


Constructors are generally stronger for mandatory invariants.

What is the difference between a field initializer and constructor assignment?

Field initializer:

C#

```
private string _currency = "USD";
```

provides an initial value as part of instance initialization.

Constructor assignment:

C#

```
_currency = currency;
```

can then overwrite that value based on constructor input.

What is the basic object initialization order?

Current Microsoft documentation summarizes class creation as:

EXAMPLE / FLOW

```
Default field initialization
    →
derived/current field initializers
    →
base-type field initializers
    →
base constructors
    →
current constructor
    →
object initializers
```

Static construction occurs beforehand when required.

What is `required`?

`required` marks a member that must be initialized by object creation unless a constructor declares that it satisfies required members.

It is an initialization contract, not full business validation.

Does `required` mean non-null?

No.

A required member must be assigned, but the assigned value can still violate nullability or business rules.

What is `SetsRequiredMembers`?

`SetsRequiredMembersAttribute` tells the compiler that a constructor satisfies the type's required-member initialization contract.

Does SetsRequiredMembers verify the constructor implementation?

No.

It informs the compiler.

The developer is responsible for actually initializing the required members correctly.

What is a primary constructor?

A constructor whose parameters are declared directly in a class or struct declaration.

Example:

```C#
public class Gift(
    long giftId,
    decimal amount)
{
}
```

Primary constructors for general classes and structs were introduced in C# 12.

Are primary constructor parameters automatically properties?

For normal classes and structs:

EXAMPLE / FLOW

No.

They remain parameters.

Records are different: positional record parameters become properties.

Are primary constructor parameters always stored as fields?

No.

The compiler only creates storage when their usage requires the value to remain available after construction.

Can another constructor exist when a primary constructor exists?

Yes, but additional constructors must ultimately call the primary constructor through `this(...)`.

Does a class with a primary constructor still get an implicit parameterless constructor?

No.

For classes, declaring a primary constructor prevents the compiler from adding the normal implicit parameterless constructor.

Are primary constructors useful for dependency injection?

Yes.

They provide concise syntax for required dependencies and Microsoft specifically documents dependency injection as a common use case.

Should constructors perform database or network calls?

Generally no.

Constructors should normally establish object state and invariants.

Heavy I/O makes creation:

EXAMPLE / FLOW

```
slow

failure-prone

hard to test

hard to reason about
```

Use explicit methods/services or async factory methods instead.

Can constructors be async?

Ordinary C# constructors cannot be declared `async`.

If initialization requires asynchronous work, use an async factory or redesign the workflow.

Should constructors call virtual methods?

Generally avoid it.

Derived state may not yet be initialized when the virtual override executes.

This can cause partially initialized-object behavior.

What is constructor injection?

Providing required dependencies through constructor parameters.

Example:

C#

```
public GiftService(
    IGiftRepository repository)
```

This makes dependencies explicit and supports DI/testability.

What does a very long constructor often indicate?

Potentially:

EXAMPLE / FLOW

```
Too many responsibilities

Too much required state

Missing value objects

Need for factory/builder

Poor service decomposition
```

It is a design signal—not automatically a bug.

What are partial constructors?

C# 14 allows instance constructors to be partial members with one defining declaration and one implementing declaration.

This is most useful with partial/source-generated code rather than ordinary classes.

Best Practices

Use constructors to establish:

```
EXAMPLE / FLOW
mandatory state

invariants

required dependencies
```

Do not add parameterless constructors when they allow invalid objects.

Validate constructor arguments where invalid values should prevent object creation.

Use specific argument exceptions instead of generic `Exception`.

Use constructor chaining with:

```
C#
this(...)
```

to avoid duplicate initialization logic.

Use:

```
C#
base(...)
```

when a base class requires constructor arguments.

Keep the main initialization path easy to understand.

Use field/property initializers for natural defaults that don't depend on caller input.

Use constructors for values that depend on caller input.

Use object initializers for optional/configuration-like state where appropriate.

Use:

```
C#
init
```

when values may be assigned during initialization but shouldn't freely change later.

Use:

```
C#
required
```

when callers must initialize members.

Remember:

```
EXAMPLE / FLOW
required
≠
validation
```

Use `SetsRequiredMembers` carefully when constructors truly satisfy required-member contracts.

Use primary constructors when they make a class clearer—not just because the syntax is shorter.

Remember primary constructor parameters aren't automatically prop

Avoid referencing primary-constructor parameters in members when an actual property/field should represent the object's state.

Use private constructors to control object creation when factory methods better express the domain.

Avoid I/O, network calls, database queries, emails, and large workflows in constructors.

Avoid virtual calls from constructors.

Use constructor injection for required service dependencies.

Treat a constructor with many dependencies as a possible SRP/design warning.

Protect constructor-created invariants after construction through proper encapsulation.

Do not expose public setters that immediately allow callers to violate the state your constructor carefully validated.

Quick Revision

Concept	Meaning
Constructor	Initializes a new instance
Parameterless constructor	Constructor with zero parameters
Parameterized constructor	Constructor receiving values
Implicit constructor	Compiler-created class parameterless constructor when no instance constructors exist
Constructor overloading	Multiple constructors with different signatures
`this(...)`	Call another constructor in same type
`base(...)`	Call direct base-class constructor
Private constructor	Restricts external creation
Static constructor	Initializes static type state
Field initializer	Gives field initial value
Property initializer	Gives property initial value
Object initializer	Caller assigns members during creation
`init`	Setter usable during initialization
`required`	Member must be initialized
`SetsRequiredMembers`	Constructor declares required-member contract satisfied
Primary constructor	Parameters declared on type declaration
Constructor injection	Dependencies supplied through constructor
Factory method	Named creation method
Invariant	Rule object must preserve
Partial constructor	C# 14 partial constructor declaration/implementation

Complete Fundraising Example

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public Gift(
        long giftId,
        long tenantId,
        long constituentId,
        decimal totalAmount)
        : this(
            giftId,
            tenantId,
            constituentId,
            totalAmount,
            currencyCode: "USD",
            campaignId: null)
    {
    }

    public Gift(
        long giftId,
        long tenantId,
        long constituentId,
        decimal totalAmount,
        string currencyCode)
        : this(
            giftId,
            tenantId,
            constituentId,
            totalAmount,
            currencyCode,
            campaignId: null)
    {
    }

    public Gift(
        long giftId,
        long tenantId,
        long constituentId,
        decimal totalAmount,
        string currencyCode,
        long? campaignId)
    {
        if (giftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(giftId));
        }

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
```

```
        }

        if (totalAmount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
            currencyCode);

        GiftId = giftId;

        TenantId = tenantId;

        ConstituentId = constituentId;

        TotalAmount = totalAmount;

        CurrencyCode =
            currencyCode
                .Trim()
                .ToUpperInvariant();

        CampaignId = campaignId;

        CreatedAtUtc = DateTime.UtcNow;
    }

    public long GiftId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public long ConstituentId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public string CurrencyCode
    {
        get;
    }

    public long? CampaignId
    {
        get;
        private set;
    }

    public DateTime CreatedAtUtc
    {
        get;
    }
```

```
    public string? Notes
    {
        get;
        init;
    }

    public bool IsCancelled
    {
        get;
        private set;
    }

    public IReadOnlyList<GiftAllocation>
        Allocations
        =>
        _allocations;

    public void ChangeAmount(
        decimal amount)
    {
        EnsureActive();

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        decimal allocated = 0m;

        foreach (
            GiftAllocation allocation
            in _allocations)
        {
            allocated += allocation.Amount;
        }

        if (amount < allocated)
        {
            throw new InvalidOperationException(
                "Gift amount can't be below the allocated amount.");
        }

        TotalAmount = amount;
    }

    public void ChangeCampaign(
        long? campaignId)
    {
        EnsureActive();

        CampaignId = campaignId;
    }

    public void AddAllocation(
        GiftAllocation allocation)
    {
        ArgumentNullException.ThrowIfNull(
            allocation);

        EnsureActive();

        decimal allocated = 0m;
```

```
        foreach (
            GiftAllocation existing
            in _allocations)
        {
            allocated += existing.Amount;
        }

        if (allocated
            +
            allocation.Amount
            >
            TotalAmount)
        {
            throw new InvalidOperationException(
                "Allocations can't exceed the gift amount.");
        }

        _allocations.Add(
            allocation);
    }

    public void Cancel()
    {
        EnsureActive();

        IsCancelled = true;
    }

    private void EnsureActive()
    {
        if (IsCancelled)
        {
            throw new InvalidOperationException(
                "The gift has been cancelled.");
        }
    }
}

public sealed class GiftAllocation
{
    public GiftAllocation(
        long fundId,
        decimal amount)
    {
        if (fundId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(fundId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        FundId = fundId;

        Amount = amount;
    }

    public long FundId
    {
        get;
    }
```

```
    public decimal Amount
    {
        get;
    }
}
```

Usage:

C#

```
Gift gift =
    new Gift(
        giftId: 1001,
        tenantId: 1,
        constituentId: 5001,
        totalAmount: 1000m)
    {
        Notes =
            "Annual giving campaign"
    };

gift.ChangeCampaign(
    2001);

gift.AddAllocation(
    new GiftAllocation(
        fundId: 3001,
        amount: 600m));

gift.AddAllocation(
    new GiftAllocation(
        fundId: 3002,
        amount: 400m));
```

Construction flow:

EXAMPLE / FLOW

```
new Gift(...)
    →
Gift overload selected
    →
this(...)
    →
main Gift constructor
    →
validate IDs
    →
validate amount
    →
validate currency
    →
initialize properties
    →
initialize CreatedAtUtc
    →
constructor completes
    →
object initializer sets Notes
    →
Gift ready
```

Primary Constructor Version

A simpler service type may benefit more from primary-constructor syntax:

```C#
public sealed class GiftService(
    IGiftRepository repository,
    ILogger<GiftService> logger)
{
    public async Task CreateAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        ArgumentNullException.ThrowIfNull(
            gift);

        logger.LogInformation(
            "Saving gift {GiftId}",
            gift.GiftId);

        await repository.SaveAsync(
            gift,
            cancellationToken);
    }
}
```

Here:

```EXAMPLE / FLOW
repository
+
logger
```

are required construction dependencies.

This is a good example of a primary constructor because there is little custom initialization logic.

Constructor Decision Flow

```EXAMPLE / FLOW
Need to create object
    →
Does object require data
to be valid?
    →
Yes
    →
Constructor parameters
or required members
```

Then:

```EXAMPLE / FLOW
Are those values
fundamental invariants?
    →
Yes
    →
Constructor often preferred
```

If values are:

EXAMPLE / FLOW

```
optional / descriptive
```

consider:

EXAMPLE / FLOW

```
init properties
+
object initializer
```

Then:

EXAMPLE / FLOW

```
Multiple creation variations?
    →
Constructor overloads
    →
Can logic be shared?
    →
this(...)
```

Inheritance:

EXAMPLE / FLOW

```
Base constructor needs input?
    →
base(...)
```

Controlled creation:

EXAMPLE / FLOW

```
Should caller NOT instantiate directly?
    →
private/internal constructor
    →
factory method
```

Static state:

EXAMPLE / FLOW

```
Need one-time type initialization?
    →
static field initializer
or
static constructor
```

Modern concise dependency holder:

EXAMPLE / FLOW

```
Simple required parameters?
    →
primary constructor may fit
```

Async creation:

EXAMPLE / FLOW

```
Requires await?
    →
```

```
ordinary constructor cannot be async
    ↓
async factory / separate service workflow
```


Complete Object Initialization Flow

EXAMPLE / FLOW

```
new Gift(...)
    ↓
Memory / default instance state
    ↓
Field / property initializers
    ↓
Base initialization chain
    ↓
Base constructors
    ↓
Gift constructor body
    ↓
Object initializer
    ↓
Fully constructed Gift
```

Then:

EXAMPLE / FLOW

```
Encapsulation
    ↓
controls future mutation
```

This last part is important.

A constructor protects only:

EXAMPLE / FLOW

```
initial state
```

Encapsulation protects:

EXAMPLE / FLOW

```
state throughout object lifetime
```

Final Mental Model

Think of construction as a gate:

EXAMPLE / FLOW

```
Raw Input
    ↓
Constructor Gate
    ↓
Validation
    ↓
Initialization
    ↓
Invariant Established
    ↓
Object Enters System
```

After the gate:

EXAMPLE / FLOW

```
Public Methods / Controlled Properties
    →
Protect Invariant
    →
Object Remains Valid
```

The most important lesson from this topic is:

> **A constructor is not merely a place to assign properties. It defines how an object is allowed to come into existence. Good constructor design ensures mandatory information is available, validates invariants, makes dependencies explicit, avoids duplicated initialization, and prevents partially valid objects from spreading through the application.**

For the Fundraising Platform, construction should help ensure that something such as a `Gift` begins life with:

EXAMPLE / FLOW

```
Valid Tenant

Valid Constituent

Valid Amount

Valid Currency

Consistent Initial State
```

instead of forcing every service and controller to repeatedly ask:

EXAMPLE / FLOW

```
"Did somebody remember
to initialize this object correctly?"
```

Next roadmap topic: Topic 13 — Inheritance.